

现代密码学课程设计说明书

学号：2454307

姓名：汤皓宇

课程：现代密码学课程设计

题目：基于 NTL 的加密、签名、证书方案、简易 PKI 与安全邮件系统设计

1. 课程设计任务

本课程设计以开源大数库 NTL 为基础，使用 C++ 实现五个相互衔接的任务。任务一包含 RSA 与 Elgamal 两种公钥体制，每种体制都要完成密钥生成、加密、解密、签名与签名验证，其中 RSA 密钥生成规定 p 、 q 均为 1024 比特素数，素性检测需要重复足够的次数，以保证通过检测的数确为素数的概率足够高；密钥与密文还需要具备文本格式的转换能力。任务二实现简单证书方案，证书内容为 $\text{Cert}(\text{Alice}) = (\text{ID}(\text{Alice}) \parallel \text{ver_Alice} \parallel s \parallel \text{ID}(\text{TA}) \parallel \text{flag1} \parallel \text{flag2})$ ，签名 s 由 TA 的私钥对 $\text{ID}(\text{Alice}) \parallel \text{ver_Alice}$ 计算， flag1 标识签名算法，0 表示 RSA、1 表示 Elgamal， flag2 标识公钥用途，0 表示加密、1 表示签名，证书以 txt 文本保存。任务三实现教材 12.3.1 节严格层次 PKI 系统的简易版本，系统包含一个自签名的根 CA、两个下级 CA 以及挂在下级 CA 之下的用户，另有一个可公开查询的证书库，查询内容为证书所有者的 ID，返回结果为该所有者的证书路径。任务四实现简易安全邮件系统，系统不依赖网络传输，发送端对邮件摘要签名后用接收端加密公钥加密，接收端解密后用发送端签名公钥验证。

五个任务之间存在清晰的依赖关系。证书方案为 PKI 提供证书对象、文本序列化与签名验证工具，PKI 利用证书库把证书组织成可验证的信任链，邮件系统则依赖证书链取得对方的可信公钥，把签名与加密组合成完整的通信协议。任务书同时要求不同对象完成的任务用独立的模块完成，这里的独立模块指独立的类与源文件，各模块编译进同一组程序，而不是相互独立的程序。第 2 节说明各模块的设计及其依据，第 3 节说明编译、测试与演示程序的使用方法，第 4 节给出实际运行结果。

2. 程序设计说明

2.1 总体结构与模块划分

项目按依赖方向分为四层。底层是 `umbra_hash` 与 `umbra_crypto` 两个库，前者提供 SHA-256，后者提供 RSA 与 Elgamal 的密钥生成、加解密与签名验证，所有大整数运算均调用 NTL。中层是 `umbra_certificate` 库，提供证书对象、证书文本序列化、公钥文本转换、证书与证书路径验证，以及 TA、CA、证书库、用户四个对象模块。上层是 `umbra_mail` 库，实现安全邮件的发送端与接收端流程。依赖方向为 `umbra_mail` 到 `umbra_certificate` 到 `umbra_crypto` 到 `umbra_hash`，单向且无环，每一层只使用下一层提供的接口。

把加密引擎独立成库，是因为证书与邮件都要复用同一套加解密和签名能力。证书文本需要同时容纳 RSA 与 Elgamal 两种公钥，证书验证要根据签发者使用的算法选择对应的签名引擎，这些需求都落在统一的引擎接口之上。源文件结构如下。

```

src/
|-- crypto/                # 加密与签名引擎
|   |-- crypto_engine.h/.cpp #
|   PublicKey/PrivateKey/Keypair/EncryptionEngine/SignatureEngine
|   |-- rsa_engine.h/.cpp   # RSA 密钥生成、加解密、签名验证、密钥文本转换
|   `-- elgamal_engine.h/.cpp # Elgamal 密钥生成、加解密、签名验证
|-- hash/
|   `-- sha_256.h/.cpp      # SHA-256
|-- certificate/
|   |-- certificate.h/.cpp   # 证书对象、txt 序列化、公钥文本转换、证书与路径
|   |验证
|   |-- trusted_authority.h/.cpp # TA 模块
|   |-- certificate_authority.h/.cpp # CA 模块
|   `-- certificate_library.h/.cpp # 证书库模块
|-- pki/
|   `-- pki_user.h/.cpp      # 用户模块，对应 Alice/Bob/Eve
|-- mail/
|   `-- secure_mail.h/.cpp   # 安全邮件模块
|-- demo/
|   `-- umbra_demo.cpp       # 演示程序

```

TA、CA、证书库、用户、邮件各自拥有独立的类与源文件，对应任务书对独立模块的要求。证书模块不定义自己的算法枚举，签名算法直接使用 `crypto::CryptoType`，其取值 0 与 1 恰好对应任务书规定的 RSA 与 Elgamal，由此省去了一套重复枚举以及枚举之间的转换代码。

2.2 RSA 加密与签名

RSA 密钥生成使用 NTL 的 `GenPrime_ZZ` 生成两个 1024 比特素数 p 和 q ，该函数内部执行多轮 Miller-Rabin 素性检测，满足任务书对检测次数的要求。公钥指数取固定的 $e = 65537$ ，这个值本身是素数，其二进制表示只有两个 1，模幂运算的开销较小。私钥指数 d 是 e 在模 λ 下的逆元，其中 $\lambda = \text{lcm}(p-1, q-1)$ 为 Carmichael 函数。选用 λ 而不是欧拉函数 $\phi(n)$ ，是因为 λ 整除 $\phi(n)$ ，对 e 的互素要求更宽松，而两种选择导出的解密指数都能正确解密。生成的公钥为 $(n = pq, e)$ ，私钥按任务书规定保存 p 、 q 、 d 三项，解密时由 p 与 q 相乘得到模数 n 。

加密时明文按 $\text{NumBytes}(n) - 1$ 字节分组，块长比模数字节数少 1，保证每个块转换出的大整数严格小于 n 。每个块计算 $c = m^e \bmod n$ ，密文文本每行一个分组，行格式为"块长-密文值"。块长信息让解密端能够精确恢复每个块的字节长度，解密时逐行读取，计算 $m = c^d \bmod n$ 后按块长调用 `BytesFromZZ` 还原明文。公钥与私钥分别通过构造函数的文本解析和 `operator std::string` 完成文本互转，公钥文本为 n 、 e 两行，私钥文本为 p 、 q 、 d 三行。密钥、证书、密文全部以文本形式落盘，便于检查与归档。

签名采用先摘要后签名的方式。对消息计算 SHA-256 摘要，再对摘要做大数运算 $s = H(m)^d \bmod n$ ，验证时恢复 $H(m)' = s^e \bmod n$ 并与本地摘要比较。摘要长度固定为 256 比特，签名长度因此只取决于模数而与消息长度无关，同时避免了直接对长消息做模幂运算。

验证与解密接收的是来自外部的签名文本和密文文本，这些数值可能超出模数范围，而 NTL 的 PowerMod 要求底数小于模数，否则抛出未捕获的错误并终止进程。实现中先把外部数值约化到模数范围内再参与运算，合法输入下这一约化是恒等操作，非法输入则只产生验证失败或解密乱码，程序不会中止。这个缺陷在测试阶段被实际触发过，用错误公钥验证签名时，签名文本中的数值约有一半的概率不小于对方模数，导致测试随机崩溃；约化之后该场景稳定返回验证失败，测试从随机失败变为稳定通过。

2.3 Elgamal 加密与签名

Elgamal 密钥生成先随机生成素数 q ，再令 $p = 2q + 1$ 并检测 p 是否为素数，重复直到两者都为素数，得到的 p 称为安全素数。要求安全素数的原因在于，Elgamal 的安全性建立在模 p 离散对数问题的难度之上， $p - 1$ 含有大素因子 q 可以排除针对群结构的一部分攻击。生成元 α 取二次剩余子群的一个生成元，实现中随机选取 r 并令 $\alpha = r^2 \bmod p$ ，该子群的阶恰好为 q 。私钥 a 在 $[1, q-1]$ 中随机选取，公钥为 $\beta = \alpha^a \bmod p$ 。私钥取值与签名指数全部落在阶为 q 的子群内，这是后续签名运算能够约化到模 q 的依据。

加密的分组方式与 RSA 相同。每个块 m 随机选取临时密钥 k ，计算 $c1 = \alpha^k \bmod p$ 与 $c2 = m \cdot \beta^k \bmod p$ ，密文行格式为"块长-c1-c2"。解密用私钥 a 计算 $s = c1^a \bmod p$ ，再计算 $m = c2 \cdot s^{-1} \bmod p$ 。临时密钥 k 每块重新选取，因此同一明文每次加密得到的密文不同，这是 Elgamal 概率加密特性的直接体现。

签名与验证遵循课本第 7 章的 Elgamal 签名方案。签名前对消息取 SHA-256 摘要并约化到模 q ，随机选取与 q 互素的 k ，计算 $r = \alpha^k \bmod p$ 与 $s = k^{-1}(H(m) - a \cdot r) \bmod q$ 。验证方程 $\alpha^{H(m)} \equiv \beta^r \cdot r^s \pmod{p}$ 两边相等则签名有效。签名分量 r 来自签名文本，用错误公钥验证时可能不小于对方模数，验证函数先对 r 取模 p 再作为幂运算底数，处理方式与 RSA 验证一致，保证错误密钥场景只返回失败。

2.4 简单证书方案

证书对象 Certificate 保存六个字段，与任务书给出的证书结构逐项对应，分别是签发者 ID、持有者 ID、持有者公钥文本、签名、flag1 与 flag2。flag1 表示签发者使用的签名算法，flag2 表示证书中公钥的用途。证书文本以 BEGIN UMBRA CERTIFICATE 与 END UMBRA CERTIFICATE 两行作为边界，各字段逐行列出，公钥与签名允许跨多行，实际格式如下。

```
BEGIN UMBRA CERTIFICATE
ISSUER:TA
SUBJECT:Alice
ALGORITHM:RSA
PURPOSE:Encryption
PUBLIC_KEY:RSA <n> <e>
SIGNATURE:
<s>
END UMBRA CERTIFICATE
```

被签名的字节串固定为持有者 ID 与公钥文本的拼接，即证书中的 SUBJECT 行与 PUBLIC_KEY 文本。签发与验证共用同一个 signed_payload() 函数，从根源上避免两侧各

自拼接产生的格式不一致。验证时以证书中的 flag1 选择签名引擎，用签发者公钥验证签名，方程成立则证书可信。

公钥文本的第一行是类型标记 RSA 或 ELGAMAL，其后才是密钥数值。证书链验证需要从上一级证书解析出下一级证书的签发者公钥，而签发者既可能是 RSA 也可能是 Elgamal，类型标记让解析无需猜测，同时让证书中能够存放任意一种公钥。TA 模块 TrustedAuthority 在构造时按指定算法生成自己的签名密钥对，grant_certificate 用私钥对 ID || ver_Alice 签名并组装证书。TA 支持 RSA 与 Elgamal 两种算法，两种算法下的签发、篡改拒绝与错误密钥拒绝均有对应的测试。

2.5 简易 PKI 系统

系统的层次关系为 CA_root 的证书由自己签发，CA1 与 CA2 的证书由 CA_root 签发，用户证书由 CA1 或 CA2 签发，形成教材 12.3.1 节的严格层次。CA 模块 CertificateAuthority 在构造时生成自己的签名密钥对，self_sign 生成自签名证书，issue_certificate 为下级 CA 或用户签发证书，签发者字段与证书的 flag1 都取自该 CA 自身的算法。演示场景中 CA1 使用 Elgamal、CA2 使用 RSA，同一套代码同时覆盖两种算法。

证书库 CertificateLibrary 只接受能够链接到受信根的证书。自签名证书必须属于受信根且能用根公钥验证，其余证书必须能由其签发者公钥验证，而签发者的证书必须已经存在于库中。校验放在入库环节，其效果是从源头保证库内证书可信，任何伪造或篡改的证书在入库时即被拒绝。查询接口按持有者 ID 返回证书路径，路径从叶证书沿签发者逐级回溯到自签名根证书，例如 Alice 的路径为 <CA_root 证书, CA1 证书, Alice 证书>。每个用户持有加密与签名两本证书，因此查询接口还提供按用途过滤的版本，供邮件系统按用途取用公钥。

证书路径验证函数 verify_certificate_path 先用受信根公钥验证根证书，再逐级用上一级证书中的公钥验证下一级证书的签名，任一级失败即判定整个路径不可信。任务书给出的使用例子中，Alice 用签名私钥对消息签名，Bob 在证书库查询 Alice 的证书路径，验证路径后用叶证书中的签名公钥验证签名，这个例子完整覆盖了证书申请、存储、查询、路径验证与签名验证的链条，测试程序按此流程逐项断言。

2.6 简易安全邮件系统

发送端需要把邮件 m 与签名 s 拼接后整体加密，而邮件内容允许包含换行，直接以行作为分隔无法可靠切分，因此 m||s 采用长度前缀编码，文本形式为 m 的字节数、换行符、m、s。解析时先读长度，按长度切出 m，余下部分即为签名。这套编码只依赖长度信息，与邮件内容是否含换行无关。

发送端的处理分四个环节。先在证书库查询接收端的加密公钥证书链并验证，这一步失败意味着接收端身份不可信，流程就此终止，交互层可以选择重新申请证书链或退出。验证通过后，用发送端签名私钥对邮件摘要签名得到 s，再从叶证书取出接收端的加密公钥，随后用该公钥加密 m||s，得到的密文即为要发送的内容。接收端的处理与之对称，先用自己的加密私钥解密得到 m||s 并按长度前缀切分，再查询发送端的签名公钥证书链并验证，最后用叶证书中的签名公钥验证签名。只有证书链可信且签名匹配，邮件内容才被接受，两个环节中任何一步失败都以异常形式终止，由调用方决定后续处理。任务书要求邮件系统不依赖网络，因此发送与接收之间只交换密文文件，协议本身与传输载体无关。

3. 程序使用说明

3.1 编译

项目使用 CMake 构建，NTL 与 GMP 可以通过scripts目录下的build_deps.sh脚本来进行编译。Debug 构建命令如下，Release 构建使用 cmake --preset release 与 cmake --build build/release，两套构建配置了相同的测试。

```
cmake --preset debug
cmake --build build/debug
ctest --test-dir build/debug --output-on-failure
```

3.2 测试程序

每个模块对应一个测试目标，共 9 个，全部注册在 CMakeLists.txt 中并纳入 ctest。

测试程序	对应模块	测试内容
umbra_crypto_rsa_engine_test	RSA 引擎	文件加解密往返、签名与验证正反例
umbra_crypto_elgamal_engine_test	Elgamal 引擎	文件加解密往返、签名与验证正反例
umbra_hash_sha_256_test	SHA-256	标准向量、流式与一次性接口一致性
umbra_certificate_test	证书	txt 序列化往返、flag 数值、公钥文本转换、签名验证
umbra_trusted_authority_test	TA	两种算法下的签发与验证、篡改与错误密钥拒绝
umbra_certificate_authority_test	CA	根自签名、为用户与下级 CA 签发
umbra_certificate_library_test	证书库	证书路径查询与验证、伪造与篡改拒绝、使用例子
umbra_pki_user_test	用户	双证书申请、签名与验证
umbra_secure_mail_test	安全邮件	发送接收往返、篡改与错误密钥场景拒绝

3.3 演示程序 umbra_demo

演示程序把五个任务串联成一次完整运行，用法为 umbra_demo [输入文件] [输出目录]。输入文件是明文来源，RSA 与 Elgamal 的加密输入以及安全邮件的邮件内容都取自该文件，缺省时使用内置示例；输出目录存放全部产物，缺省为当前目录下的 demo_output。任务书要求控制台程序在输入量大时优先使用文件输入，输入文件参数即为此设计。

运行依次演示 RSA 加密签名、Elgamal 加密签名、简单证书方案、简易 PKI 系统与安全邮件系统五个部分，每个部分把关键结果打印到屏幕，同时把产物写入输出目录。主要产物的格式

如下。

产物	格式
rsa/rsa_public_key.txt	n 换行 e
rsa/rsa_private_key.txt	p 换行 q 换行 d
rsa/rsa_ciphertext.txt	每行"块长-密文值"
rsa/rsa_signature.txt	s
elgamal/elgamal_public_key.txt	p 换行 alpha 换行 beta
elgamal/elgamal_ciphertext.txt	每行"块长-c1-c2"
certificates/*.cert	第 2.4 节的证书文本格式
pki/root.cert 等	各 CA 与用户证书
pki/alice_chain.txt	Alice 的证书路径
mail/mail_ciphertext.txt	安全邮件密文
mail/mail_recovered.txt	接收端恢复的邮件内容

4. 测试与运行结果

测试在 debug 与 release 两种构建下均全部通过。debug 构建下 ctest 的输出如下。

```
● (base) tonhie:debug/ (mainx) $ ctest
Test project /Users/tonhie/Projects/Umbra-Core/build/debug
  Start 1: crypto.rsa_engine
1/9 Test #1: crypto.rsa_engine ..... Passed    0.01 sec
  Start 2: hash.sha_256
2/9 Test #2: hash.sha_256 ..... Passed    0.01 sec
  Start 3: crypto.elgamal_engine
3/9 Test #3: crypto.elgamal_engine ..... Passed    0.08 sec
  Start 4: certificate.certificate
4/9 Test #4: certificate.certificate ..... Passed    0.08 sec
  Start 5: certificate.trusted_authority
5/9 Test #5: certificate.trusted_authority ..... Passed    0.17 sec
  Start 6: certificate.certificate_authority
6/9 Test #6: certificate.certificate_authority ... Passed    0.05 sec
  Start 7: certificate.certificate_library
7/9 Test #7: certificate.certificate_library ..... Passed    0.53 sec
  Start 8: pki.pki_user
8/9 Test #8: pki.pki_user ..... Passed    0.04 sec
  Start 9: mail.secure_mail
9/9 Test #9: mail.secure_mail ..... Passed    0.68 sec

100% tests passed, 0 tests failed out of 9

Total Test time (real) = 1.66 sec
```

证书库测试覆盖了证书链的查询、验证与反例。Alice 的证书路径长度为 3，顺序为 CA_root、CA1、Alice，路径验证通过。未知 CA 签发的证书、非根的伪造自签名证书、错误 CA 密钥签发的证书均被证书库拒绝入库。篡改叶证书或中间 CA 证书之后，路径验证失败。Bob 查询 Alice 的证书路径并用叶证书中的公钥验证其签名的场景得到预期结果。安全邮件测试覆盖了完整往返与各失败场景，Alice 发送的邮件经 Bob 接收后与原文逐字节一

致，篡改密文、使用错误的接收端私钥、发送端无证书、证书链不以受信根为根这四种情况均被拒绝。测试输出中每条断言逐项以 PASS 列出，任何断言失败都会使测试返回非零退出码。

演示程序输出如下：

4.1 RSA加密与签名演示

```
=====
[任务1] RSA 加密和签名
=====
```

```
生成 RSA 密钥对 (p, q 均为 1024 比特素数) ...
```

```
RSA 公钥 (n, e):
```

```
2171942135889035272230272492485367678528152676157085888962564867506130477758941790607709813859793180172508239251759937
1314728259669901084626908026920325240183354007202548079538992648922238571297365924467805981375451131983635155634770574
2020076854732156767128545937307833629497301221424514024328686100569802881346745052978381695540772190528222433101089714
4204717804965456416918708520182781600598652503413625251415631471313764712365870546438637315571549468028913686576619044
1148862972399090826816072768263284859890162299609010752619372090532132044091533631765000867702097826441355927438463488
027579020090201272190055873
65537
```

```
RSA 私钥 (p, q, d):
```

```
1212623492274913449977744040655614298430679742242796903145438449722653356356520966467354689503174743119100713731318202
1714977347567022842102596721697040324053466790128533612884824513935515172340436964629662375572572466563596732206690575
3018690298681311414717716512028127662529816103103577626923190222732371353
1791110059903601988252038147998105256978650066686354836909994097722601723106043519452505123922489310829111304963898640
1210285493562528925371837808113228715838898960250253418585365335588988462822192471560688996552598763340348650172951847
8466014354489742271653180977028006780091741129730482344838077781409716841
7963710503290281458060858445522893222917055832286307568514035394688849868097009510399204545073901783655854401211497518
8472606326177231649844301674915759879397286539371230045824800243160564699067046579025798820887756641525665317286958038
6765101670691933886827129541100026262541792933889416971270935043821408237573060464198969462011243991187314669718325606
7498391094801330019296240313889557943733052549070348448626314320644698236913164038417584833113153175365676008944963402
1207051522722054980797412925795639358929942679391165576624721294789059422454811839970177268930336162376500645459855854
2960865970893184660867393
```

```
明文长度：223 字节
```

```
RSA 密文：
```

```
223-499737691999755684942652368527407221749857436479155394621507355188374400032600568478138922017388357886156272461732
0120168853403364654953665101339049665964721142006899750833012984598310309578724179783540954058742497584247191877010190
2870433315987625058408845601001058745855964021947439317905147727109853022266952118474983992709267450245418509155918664
2469720359683878702807289311237374497620090019464238985458780276998031723281218906123442365925691679521010363951962295
66865959127465817768036703451536400989852108090517420825344842707561297164014280693810109310929109086173100003355411839
200469242217395690030830340138
```

```
RSA 解密结果与明文一致：是
```

```
RSA 签名验证（对明文 Hash 签名）：通过
```

```
产物目录：demo_output/rsa
```

4.2 Elgamal加密与签名演示

```
=====
[任务1] Elgamal 加密和签名
=====
```

```
生成 Elgamal 密钥对 (p = 2q + 1 安全素数, 512 比特) ...
```

```
Elgamal 公钥 (p, alpha, beta):
```

```
7342016760546615851133625876568500308060085509679757310383124317541357271265637067202408682179728165530845126137975983
938950877884334897246940162534064799
6842668502348460504916058066625933755831020591469494132139903808449491669773078789290504664956608877058292074598140531
05770527459983157555200709473327738
2186438981452004918635523348600792767928079905535237468692818383010473456356522568293872591654860976998959078702843523
989744637293151633946983108661131482
```

```
Elgamal 私钥 (p, alpha, a):
```

```
7342016760546615851133625876568500308060085509679757310383124317541357271265637067202408682179728165530845126137975983
938950877884334897246940162534064799
6842668502348460504916058066625933755831020591469494132139903808449491669773078789290504664956608877058292074598140531
05770527459983157555200709473327738
2178093482114864961539691769231717133620934957332561512157945770926649906049471026594986325532111590783340593775817104
637097584146687269656973642387474641
```

```
明文长度：223 字节
```

```
Elgamal 密文（每个分组：长度-c1-c2）：
```

```
63-1551013625141301137732716328648505228332862510871656311198312877804451224492180365071033616344169528990576410270269
819063700756324142063205727537787551734-706691334423535511907811181318731294860008360250721202120568755025518078510824
253248375061457921771814226346332802827180218849459371507013345408594200257
63-1022624316970603469023806288806791925845101206007718959883490116414129378467743250990187621670035096643063961485732
518911705226392345550007349801707393009-678196348514888011268303711918938529756211234911756704590436326486938195058438
1088189886271572806715715580927235145006169347743243086853468434144125131298
63-5511325663276009571615749324702876569026812656802157851285531837706594361522368596509091841357890332764493286230316
274211225429911450061105089606290752299-618121763709919583593703023336120591253685906501519261024635036701295354150228
3669274899350014276525770378782211731962409345690789659698384782279648188549
34-3799224605717826265838555246684385662462075570745097807913730633320283752875888076893413781688086071261041198783837
611015338881114994423855734622078457144-786005010007438800009514972105261065496551473961332596812019854389927777948306
218059783792022482250883383963071446522889085899314526494776057745964178703
```

```
Elgamal 解密结果与明文一致：是
```

```
Elgamal 签名验证：通过
```

```
产物目录：demo_output/elgamal
```

4.3 签发证书

```
=====
TA(RSA) 为 Alice 颁发的证书 (flag1=RSA, flag2=Encryption):
BEGIN UMBRA CERTIFICATE
ISSUER:TA
SUBJECT:Alice
ALGORITHM:RSA
PURPOSE:Encryption
PUBLIC_KEY:RSA
1199636604814121422486384782823427203362542757126263253655120831658035731125578348360223088500636756385400763670313143
4574410706501253377023036338904000394470746992825968591161141968952809501804813620699148281226424615421053108219151481
8133880635616691270352545793722606867638352801653413619150322579058148763022577871380692296989676187211082305019246491
8020111767531181507837690707224198432811917425688646135502261198822705575496712671745221573275899818000733048910560233
3028385416183584371731093000266316993617397639977419919030737745821407294155583718649339750595947522685977501034340455
420819649142943938244219399
65537
SIGNATURE:
2647976367000069234680153720688941222286375889008126683924809305956395103437615555027231547127399876217439870636357276
8903969857261034647759400601114220659318947263028501557223627523141855572183213064827034788641124005601032526343949449
8932639614265073309172946349416436147413469377981648334800002377879076762176504843574758816313427702501385236041083060
0890800790632874115985763949267320113975553071191206915098630340986439170395422871071340592904263817150756873899106720
2153895618227904024371449337670782718330731817304235602122422261296865426873150732542619124784260024671557259025353993
342980004126749890089087278
END UMBRA CERTIFICATE

证书签名验证: 通过
txt 证书文件重新解析: 成功
TA(Elgamal) 为 Alice 颁发的证书 (flag1=Elgamal, flag2=Signature):
BEGIN UMBRA CERTIFICATE
ISSUER:TA
SUBJECT:Alice
ALGORITHM:Elgamal
PURPOSE:Signature
PUBLIC_KEY:ELGAMAL
8789428590547111295487108845150509971794638370573843062532539772964841905847308993770463948015569891935256240531443010
074710281693764838971331861189120519
8305294708443760797789982848425048841396462312776868180572509747011931039918834652616519651978198018297055116220734307
754140302692133881011470814566166892
2352224754342148384626985250134449184620616596873274265291880080273426983686477162819483121436911299830816208352233687
50114099515333538680288609586087633
SIGNATURE:
8935505095095162182838360089059274682678075913948422718263150710441047477423010627225646985197028868110398072701403966
877884261923920666647806479975211776
1582286554224385328294082127756658828197273405563764366913905939289921963982499217151704299035025878601185245595785756
637961971660814308128213405315134284
END UMBRA CERTIFICATE

证书签名验证: 通过
产物目录: demo_output/certificates
```

4.4 简易PKI系统

```
=====
[任务4] 简易 PKI 系统 (严格层次结构)
=====
创建根 CA (CA_root) 并生成自签名证书...
创建下级 CA (CA1, CA2), 证书由 CA_root 签发...
初始化证书库并存储各 CA 的证书...
用户申请证书: Alice -> CA1, Bob -> CA2, Eve -> CA1
证书库中证书总数: 9

使用例子: Bob 在证书库中查询 Alice 的证书路径...
Alice 的证书路径 (证书链):
- CA_root (签发者: CA_root, flag1=0 [0=RSA,1=Elgamal], flag2=1 [0=加密,1=签名])
- CA1 (签发者: CA_root, flag1=0 [0=RSA,1=Elgamal], flag2=1 [0=加密,1=签名])
- Alice (签发者: CA1, flag1=1 [0=RSA,1=Elgamal], flag2=1 [0=加密,1=签名])
证书路径验证: 通过

Alice 发送消息: Bob, please review the design document by Friday. -- Alice
Alice 的签名:
1017016444723394070545310989884511216266357282637321672341977284272179013043453046359327112332186825791851985544614501
1751255280034614197968146660795007393526271424962305235091727522350721495542749432415709902603284725928957825760073083
245398165560227234671455126579702642088645820343253525511566250113546186326524671395705787801327656531393785987327298
7409863935702014681429234801982389749669856531722539075641887238180118489224390538258119512070626185735703076962123742
1913609717968239269993686752119247248196162416717154594315965798179484396596793227158850978811189134351805122153526959
847623884776985728253774140
Bob 用 Alice 证书中的公钥验证签名: 通过
```

4.5 简易安全邮件系统


```

=====
[任务5] 简易安全邮件系统（无网络传输）
=====

邮件内容 m (223 字节):
Hello Bob!
This is Alice. Here is the secret plan for our course design:
  1. RSA + Elgamal encryption and signature
  2. Simple certificate scheme
  3. Simple PKI system
  4. Simple secure mail system
Best regards, Alice.

[发送端 Alice]
  步骤1: 在证书库中查询 Bob 的加密公钥证书链并验证...
  Bob 的加密证书路径:
    - CA_root
    - CA2
    - Bob
  证书链验证: 通过
  步骤2: 用发送端签名私钥对邮件 Hash 值签名 (s = sig(H(m)))
  步骤3: 从 Bob 的证书中取出加密公钥
  步骤4: 用 Bob 的加密公钥加密 m||s
  步骤5: 发送密文 c (4384 字节):
63-1143086923652093611228029851458319432293453723208085288859080945169498036872761311072330530498304127388954942291171
5871306769246441288020039437541542013807-98096712404099344646629917608120615427733143590557434811053297056120073448283
38454761834472894758410397261345648072605096724746807831749765942321917391209
63-7293983572250327293166276088321049256725561656622652918658024458185069224548991355043646122992016865917784710484227
148989861554872931679138490567348632919-943978346627134011875169922038581893041997916731072652610804418462930467955483
4544113657192689605121020769644399663797451676155554454867523998567424322211
63-9253147392108128887171083846754623955851082700297233711622831754798282122608594022213889937098279427735533128617092
022417402881681887027352061463945141283-15205573222650155730872283871222601310895519938091184798800128042734231463579
040874617013997750782811034209572813559659619389370881230450848074964099549
63-9597953191595772500525381002653917659681751375366712978615610725168343462632930811641490345081862378496667345783643
498654521508930175648009803538264716573-109993289646299099274578251378508314264048711230641910004329466776441946190541
8438722533198262366146874618878791034423494432018419500246808397902635964445
63-2516515675230081566713687109627443487969402745885141373514692500178933733723776570985310023386631163144588691720687
595561244715501300359679398934602761875-113579419600619763713659940646479389731895936105811568163149164460026894074835
58392403817698161351980167098359347995161774226234707249902497313175788136948
63-7467102738639706947762752707171298481252628506205076933942774905369551938754282401067358510734342542024797970434767
294362753661750303494118929580625323363-956583194905837630833717016036484193313983455539154375544865819302206676653602
23968148344265646341862700578293868959701443141010010128805103049945751800
63-5834606948017894713608648948277123181514989857828887423726699252583288751877949407932763553853662088122180068595930
256225818514621344382032317370418244308-102823764485704148214111963103267540923677046857930232330421813699843248154195
88264820667244230822500741723126126035819252055216262043165957628094889741569
63-9015002714767410102213374581679696897285511736369815989647254109462403644296621282390430117255285776326197498347019
300527913395482087752514895334397715370-352466712842093567963804700921552721046292379384044972555102615580120950325061
160710943305989699822130452397028069823830071698957461673447669809040364999
63-8954174014514329134594403913130729420518905929459560249137323492201164033732958827582342671583152807009337573235558
390896707157856775792387357938425888622-144117355424878379461164149924384134924117096410449465114639337192290367376520
8799077805948808340917830926921523360057053964413316879345471033944128606557
63-9638338788998909074112720651713128676604792255471863822902319164768492588812560987484646865037332213042634304873736
163880163936993796651220053708437152805-850889835624387745037287171492843653367724930806200178194511462134303824040876
8862012451716128511939095529719964111242542134369787336760392323337013168005
63-1029283862889079587465626126579719380062427225135260187235884650153125952236104437025875124120409942255833614107952
2546126276960370408890612016262644720539-3845097492222258451271920822238191250572920326602151007279974232333220943588
57042130023908814093045121300165826607219818875364191836963255351223201955529
63-5764158161003142120415550560322981708478258634390569040744141907446678512517499197246076209022429972987634526540458
258210047827685126665313429896282836300-184277403934778208723901481298671294328921315424376413312283780520150194702089
7640398445614075898485227742129863320818810920960824759812218150213428968013
63-3335132021862447639184653437674166645520360649483396561003049807104058729098812412144503352581601825787078232126204
75520211097708091488040012024880987103-3408881472964591257617782477219864754978877340601692132126309869189108114937314
808312046158636428214321170013000209504269835528891600225152303807394425807
24-9788778394701382499631280117816000163409665765941645559278176720628984940163194053285175826395487698849382635147936
535609950464178756029134864376031449579-513850624379166304885045714733017098273083518338962139841053704983912955569082
6662780475901716272812682567858308774564009770330959716480619819269207876483

[接收端 Bob]
  步骤1: 用接收端加密私钥解密 c, 得到 m||s
  步骤2: 在证书库中查询 Alice 的签名公钥证书链并验证...
  步骤3: 从 Alice 的证书中取出发送端签名公钥
  步骤4: 用发送端签名公钥验证签名
Bob 解密并验证后的邮件内容:
Hello Bob!
This is Alice. Here is the secret plan for our course design:
  1. RSA + Elgamal encryption and signature
  2. Simple certificate scheme
  3. Simple PKI system
  4. Simple secure mail system
Best regards, Alice.

邮件完整性与发送者身份验证: 通过
产物目录: demo_output/pki / demo_output/mail

```

5. 总结

五个任务全部实现并通过测试。RSA 与 Elgamal 的密钥生成、加解密、签名验证基于 NTL 完成，RSA 密钥尺寸为任务书要求的 1024 比特。简单证书方案、严格层次 PKI 与安全邮件

系统分别由 TA、CA、证书库、用户、邮件模块实现，模块划分与任务书要求一致。

设计过程中有几处决定值得说明。Elgamal 使用安全素数与二次剩余子群，使私钥与签名指数全部落在阶为 q 的子群内，签名运算得以约化到模 q 。证书的 flag1 直接复用 `crypto::CryptoType`，省去重复枚举及其转换，取值仍与任务书规定的 0、1 一致。公钥文本携带类型标记，解决了 RSA 与 Elgamal 混合部署时证书链解析的二义性。证书库在入库环节完成信任校验，把不可信证书挡在库外，查询得到的路径因此可以直接使用。`m||s` 采用长度前缀编码，对含换行的邮件同样可靠。引擎层对来自外部的数值先约化再运算，这一处理由测试中实际出现的随机崩溃驱动，其结果是从错误公钥、篡改密文等输入只会得到失败结果而不会中止进程。

现有实现仍有两处可扩展的空间。证书没有有效期与吊销机制，证书库也没有做并发访问与持久化存储，这些在真实 PKI 中都是必要组成部分。另外，密钥生成耗时随位数增长明显，Elgamal 2048 比特密钥生成约需 90 秒，演示程序因此采用 512 比特的 Elgamal 密钥以控制运行时间，RSA 仍保持任务书要求的 1024 比特。除任务书指定的 NTL 及其依赖的 GMP 外，程序未使用其他加密库。